
Kubernetes Rollbacks

Goal: Understand how Kubernetes handles **rollbacks for Deployments**, how to trigger them, and best practices.

Step 1: What is a Rollback?

- A **rollback** is the process of returning a Deployment (or StatefulSet) to a **previous version**.
- Useful if a new deployment fails or behaves incorrectly.
- Kubernetes keeps a **history of previous Deployment revisions** automatically.

Think of it like “undo” for your Deployment.

Step 2: How Rollbacks Work

1. When you **update a Deployment**, Kubernetes creates a **new ReplicaSet**.
2. Old ReplicaSets are kept around (by default) for history.
3. If something goes wrong, you can **rollback** to an old ReplicaSet.

Automatic Rollback vs Manual Rollback

Type	Description
Automatic	Can be triggered by Readiness/Liveness Probes failure in combination with CI/CD pipelines. Kubernetes itself does not auto-rollback, but you can automate it.
Manual	Use <code>kubectl rollout undo deployment <name></code> to revert to a previous version.

Step 3: Lab Example — Rollback a Deployment

Step 3.1: Create Initial Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
        version: v1
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

```
kubectl apply -f nginx-deploy.yaml
```

```
kubectl get pods -l app=nginx --show-labels
```

- Two pods are running with **version=v1**.

Step 3.2: Update Deployment to New Version

```
# nginx-deploy.yaml
metadata:
  name: nginx-deploy
spec:
```

```
replicas: 2
selector:
  matchLabels:
    app: nginx
template:
  metadata:
    labels:
      app: nginx
      version: v2
spec:
  containers:
  - name: nginx
    image: nginx:1.26
```

```
kubectl apply -f nginx-deploy.yaml
```

```
kubectl get pods -l app=nginx --show-labels
```

- New pods are created with **version=v2**.
 - Old pods with **version=v1** are gradually terminated (rolling update).
-

Step 3.3: Trigger a Rollback

```
kubectl rollout undo deployment nginx-deploy
```

- By default, this **rolls back to the previous ReplicaSet (v1)**.
- Verify:

```
kubectl get pods -l app=nginx --show-labels
```

```
kubectl rollout status deployment nginx-deploy
```

Step 3.4: Rollback to a Specific Revision

1. List Deployment revisions:

```
kubectl rollout history deployment nginx-deploy
```

Example output:

```
REVISION  CHANGE-CAUSE
```

```
1         <none>
```

```
2         <none>
```

2. Rollback to a specific revision:

```
kubectl rollout undo deployment nginx-deploy --to-revision=1
```

Step 4: Check Deployment Status

- Check rollout history:

```
kubectl rollout history deployment nginx-deploy
```

- Watch rollout progress:

```
kubectl rollout status deployment nginx-deploy
```

- Get events for debugging failed rollouts:

```
kubectl describe deployment nginx-deploy
```

Step 5: Best Practices for Rollbacks

Tip	Explanation
Enable readiness/liveness probes	Detect failed pods quickly
Use version labels	Easier to identify revisions and track changes
Check rollout history	Know which version to roll back to
Limit number of old ReplicaSets	Use <code>spec.revisionHistoryLimit</code> to save cluster resources

spec:

```
revisionHistoryLimit: 5
```

- Keeps the last 5 revisions for rollbacks.

Step 6: Lab Exercise — Combined Rollback Scenario

1. Deploy **v1** of an NGINX app
2. Update Deployment to **v2** with a faulty config (simulate failure)
3. Observe pods failing to start (`kubectl get pods`)
4. Trigger **rollback** to v1
5. Verify pods are back to **stable version**

This lab demonstrates **why labels, selectors, and versioning are important** in production.

✓ Key Takeaways

- Kubernetes automatically maintains **Deployment history** via ReplicaSets
- **Rollbacks are triggered via `kubectl rollout undo`**
- Labeling pods (e.g., `version=vX`) makes rollbacks **easy and transparent**
- Always test rollbacks in a **safe environment** before production
